

COURSE CODE/TITLE:	CPE 010	SECTION:	CPE21S2
DATE PERFORMED:	July 10, 2026	DATE SUBMITTED:	July 15, 2026
NAME:	Paula Esguerra	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. ____ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A:

tryhoa4.cpp tryhoa3.cpp tryhoa.

```
1 #include<iostream>
2
3
4 int main () {
5     // #1 create a function that will take two numbers and display the sum
6     int sumOfTwoNum(int fNum, int sNum);
7
8     std::cout << "the sum of the two numbers is: " << sumOfTwoNum(6,7) << "\n\n";
9
10
11     return 0;
12 }
13 // #1
14 int sumOfTwoNum(int fNum, int sNum) {
15     int sum = fNum + sNum;
16     return sum;
17 }
```

the sum of the two numbers is: 13

Analysis: with my professor's guide, I created a function, the two variables are called fNum and sNum, under int main, we created where the numbers given inside the int main can be summed and return their value.

tryhoa4.cpp tryhoa3.cpp tryhoa.

```
1 #include<iostream>
2
3
4 // #2 Create a function that will return whether variable A is greater than variable B
5 void agreaterb(int varA, int varB);
6 agreaterb(8, 9);
7
8
9
10
11 // #2
12 void agreaterb(int varA, int varB) {
13     if (varA > varB) {
14         std::cout << "Comparison between A and B: " << varA << " is greater than " << varB << "\n\n";
15     } else {
16         std::cout << "Comparison between A and B: " << varA << " is less than " << varB << "\n\n";
17     }
18 }
```

Comparison between A and B: 8 is less than 9

Analysis: I created a function that can detect if a is greater than b, I used void so I don't have to return anything, then used an if-else loop; the numbers are inside the int main function.

```
1 #include<iostream>
2 class Triangle{
3
4 // #3 Create a function that will take two Boolean values and display the result of all logical operations then return true if it was a success.
5 bool bulV(bool a, bool b):
6 bulV(0,1):
7 std::cout << "Function returned success status: " << std::boolalpha << bulV << "\n\n";
8
9 }
10
11 // #3
12 bool bulV(bool a, bool b) {
13     std::cout << std::boolalpha;
14
15     std::cout << "Logical AND (" << a << ", " << b << "): " << (a && b) << "\n\n";
16     std::cout << "Logical OR (" << a << ", " << b << "): " << (a || b) << "\n\n";
17     std::cout << "Logical NOT (!" << a << "): " << (!a) << "\n\n";
18     std::cout << "Logical NOT (!" << b << "): " << (!b) << "\n\n";
19
20     return true;
21 }
```

Logical AND (false, true): false

Logical OR (false, true): true

Logical NOT (!false): true

Logical NOT (!true): false

Function returned success status: true

Analysis: At this last task in ILO A, I needed to create a function where all logical operations are displayed and return true, and then I searched how to put the logical operations, where AND is "&&," OR is "||," and NOT is "!", then the numbers to be displayed either true or false are in the int main function. I also used "boolalpha" so when true or false are displayed, they won't be printed as 1 or 0, but instead as true or false.

ILO B:

tryhoa4.cpp tryhoa3.cpp tryhoa2.cpp tryhoa1.cpp main.cpp

```
1 #include<iostream>
2 class Triangle{
3
4     private:
5         double totalAngle, angleA, angleB, angleC;
6     public:
7         Triangle(double A, double B, double C);
8         void setAngles(double A, double B, double C);
9         const bool validateTriangle();
10 };
11
12
```

```
Triangle set1(40,50,110);
if(set1.validateTriangle()){
    std::cout<<"the shape is a valid triangle \n.";
}
else{
    std::cout<<"the shape is a NOT valid triangle \n.";
}

return 0;
```

```
Triangle::Triangle(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}

void Triangle::setAngles(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}
```

the shape is a NOT valid triangle

Analysis: Guided by a professor, we did the ILO B, where a user will input some numbers, three numbers to be exact to measure a triangle, and now with the given numbers, the code will validate if the given numbers are to form a triangle, where 40, 50, and 110 cannot form a triangle, so the output gave me "not valid triangle". We also used double so we could use numbers with decimal points.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```

1  #include<iostream>
2
3
4  int main () {
5      int a1 = 9;
6      int a2 = 87;
7      int temp;
8
9      std::cout << "This is before the swap (a1,a2): " << "(" << a1 << ", " << a2 << ")" << std::endl;
10
11     temp = a1;
12     a1 = a2;
13     a2 = temp;
14
15     std::cout << "This is after the swap (a1,a2): " << "(" << a1 << ", " << a2 << ")" << std::endl;
16
17     return 0;
18 }

```

```

C:\Users\Paramount Laptop 5  X  +  v
This is before the swap (a1,a2):(9,87)
This is after the swap (a1,a2):(87,9)

-----
Process exited after 0.3747 seconds with return value 0
Press any key to continue . . .

```

Analysis: In this first part of the supplementary, I was given to an instructions to swap two numbers from it's given name, so a1 has an equivalence of 9 and a2 has 87, so using temp, temp will equivalent a1, now I can equivalent a1 to a2, then a2 to temp, temp has the number 9, and a1 has the number of a2, so now a2 has the number 87.

```

1  #include<iostream>
2
3
4  int main () {
5
6      double kelvinF;
7      double faren;
8
9      std::cout << "Enter Temperature in Kelvin: " ;
10     std::cin >> kelvinF;
11
12     faren = (kelvinF - 273.15) * 9 / 5 + 32;
13
14     std::cout << "The temperature in Kelvin: " << kelvinF << "\n";
15     std::cout << "The temnperature in Fahrenheit: " << faren << "\n";
16
17
18     return 0;
19 }

```

```

C:\Users\Paramount Laptop 5  X  +  v
Enter Temperature in Kelvin: 23
The temperature in Kelvin: 23
The temnperature in Fahrenheit: -418.27

-----
Process exited after 3.232 seconds with return value 0
Press any key to continue . . . |

```

Analysis: Using double, I can use decimal points in kelvinF and faren, kelvinF will be given by the user, where I inputted 23, so now faren has the formula to change kelvin to fahrenheit. So when I did print, kelvinF, it printed 23, because of my input, and faren computed the number 23, and outputs -418.27 as the answer.

```

1 #include <iostream>
2 #include <cmath> // Fixes: 'sqrt' and 'pow' is not a member of 'std'
3
4 // Declaring the function prototype above main so the compiler knows it exists
5 double calDistance(double x1, double x2, double y1, double y2):
6
7 int main() {
8     double x1, x2, y1, y2;
9
10    std::cout << "Enter the coordinates for point (x1, y1): ";
11    std::cin >> x1 >> y1;
12
13    std::cout << "Enter the coordinates for point (x2, y2): ";
14    std::cin >> x2 >> y2;
15
16    // Fix: Call the function normally without 'void' and store its return value
17    double d = calDistance(x1, x2, y1, y2);
18
19    std::cout << "The Distance Between The Two Points Is: " << d << "\n";
20
21    return 0;
22 }
23
24 // Fix: Change return type from 'void' to 'double' since it returns a calculated value
25 double calDistance(double x1, double x2, double y1, double y2) {
26     // Fix: Removed the broken 'std;' nested typo
27     return std::sqrt(std::pow(x2 - x1, 2) + std::pow(y2 - y1, 2));
28 }
29

```

```

C:\Users\Paramount Laptop 5 x + v
Enter the coordinates for point (x1, y1): 25 85
Enter the coordinates for point (x2, y2): 52 66
The Distance Between The Two Points Is: 33.0151

-----
Process exited after 16.53 seconds with return value 0
Press any key to continue . . . |

```

Analysis: Using cmath, it allowed me to use sqrt and pow, I needed them for the formula to know the distance between two points. I programmed it where the user can input the numbers and the code will just formulate the number to find the distance between the two points.

```

1  #include <iostream>
2  #include <cmath>      // Required for sqrt() and pow()
3  #include <string>      // Required for std::string
4
5  class Triangle {
6  private:
7      double totalAngle, angleA, angleB, angleC;
8
9  public:
10     // Constructor declaration
11     Triangle(double A, double B, double C):
12
13     void setAngles(double A, double B, double C):
14
15     // Use 'const' matching your original setup
16     const bool validateTriangle():
17
18     // a. Function to compute for the area of a triangle (Heron's Formula)
19     double computeArea(double a, double b, double c) {
20         double s = (a + b + c) / 2.0;
21         return std::sqrt(s * (s - a) * (s - b) * (s - c));
22     }
23
24     // b. Function to compute for the perimeter of a triangle
25     double computePerimeter(double a, double b, double c) {
26         return a + b + c;
27     }
28
29     // c. Function that determines whether the triangle is acute, obtuse, or 'others'
30     std::string determineType() {
31         if (!validateTriangle()) {
32             return "others (Invalid angles)";
33         }
34         // Check if any single angle is exactly 90 degrees (Right-angled)
35         if (angleA == 90 || angleB == 90 || angleC == 90) {
36             return "others (Right-angled)";
37         }
38         // Check if any single angle is greater than 90 degrees
39         if (angleA > 90 || angleB > 90 || angleC > 90) {
40             return "obtuse-angled";
41         }
42         // If all angles are less than 90 degrees
43         return "acute-angled";
44     }
45 };
46
47 // Dummy validation function matched from your main placeholder logic
48 bool bulV(bool a, bool b) {
49     return a || b;
50 }
51
52 int main () {
53     // Standardizing your pointer/boolean display log
54     std::cout << "Function returned success status: " << std::boolalpha << bulV(0, 1) << "\n\n";
55
56     // Testing your test inputs (40, 50, 110)
57     Triangle set1(40, 50, 110);
58
59     if (set1.validateTriangle()) {
60         std::cout << "the shape is a valid triangle.\n";
61     } else {
62         std::cout << "the shape is a NOT valid triangle.\n";
63     }
64
65     // Displaying the classification result
66     std::cout << "The triangle is classified as: " << set1.determineType() << "\n\n";
67

```

```

68 // Example execution for Perimeter and Area using sample side dimensions
69 // Example sides matching 40-50-110 setup rules: Side A=4.5, Side B=5.4, Side C=8.2
70 double sideA = 4.5, sideB = 5.4, sideC = 8.2;
71 std::cout << "--- Triangle Measurements ---\n";
72 std::cout << "Perimeter: " << set1.computePerimeter(sideA, sideB, sideC) << "\n";
73 std::cout << "Area : " << set1.computeArea(sideA, sideB, sideC) << "\n";
74
75 return 0;
76 }
77
78 // Constructor Implementation
79 Triangle::Triangle(double A, double B, double C) {
80     setAngles(A, B, C);
81 }
82
83 // Member Function to Set Angles
84 void Triangle::setAngles(double A, double B, double C) {
85     angleA = A;
86     angleB = B;
87     angleC = C;
88     totalAngle = A + B + C;
89 }
90
91 // Member Function to Validate Triangle Rules (Sum of angles must equal 180)
92 const bool Triangle::validateTriangle() {
93     // Angles must also all be greater than zero to form a flat plane
94     if (angleA <= 0 || angleB <= 0 || angleC <= 0) {
95         return false;
96     }
97     return (totalAngle == 180.0);
98 }
99

```

Function returned success status: true

the shape is a NOT valid triangle.

The triangle is classified as: others (Invalid angles)

--- Triangle Measurements ---

Perimeter: 18.1

Area : 11.3028

Process exited after 0.1821 seconds with return value 0

Press any key to continue . . . |

Analysis: In the main template, where I got the original code, using the third task on ILO A, printing bluV directly gets the raw internal address of the function. Then changing bluV(0,1) outputs the true computed evaluation of true or false statement instead. Then in the computeArea, the function will compute the area of the triangle using Heron's Formula. At B, the function will compute the perimeter which turned out as success, the triangle has 18.1 as the perimeter. Then at C, the function determines whether it is acute, obtuse or others, but since the triangle has the measurements of 40, 50, 110, it isn't anything but invalid angles. Then bluV functionality incorporates the truth table steam tracking inside the bluV.



3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Google, Gemini.

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Checking, Idea Generation, and explanation of concepts.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: My work had still holes, where I used AI to check them for me. As I came up with some ideas, I laid them there so I would have a generalization of what to do and how to do it. I asked, then I worked through it.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.